

FinTechCo Demo

Step-by-Step Run Sheet

40-minute mock customer meeting | ~13-minute live demo | 5 prompts

Audience

Role	Mindset	What They Care About
CTO	Skeptical, security-first, 20+ yrs fintech	Compliance, audit trails, control, no vendor lock-in
Head of DT	Champion, wants productivity wins	Scale to 180 engineers, velocity metrics, onboarding speed

Key Framing — Say This, Not That

Don't Say	Say Instead
"AI coding assistant"	"Agentic coding tool — it reads, reasons, acts, validates"
"It writes code for you"	"It's an extension of your team — unlimited capacity, same rigor"
"Like Copilot but better"	"Copilot autocompletes lines. Claude Code executes multi-step workflows across your whole repo"
"It can help with..."	"Watch — it's about to read 7 files, edit 2, run 13 tests, and open a PR. One prompt."

Pre-Flight Checklist

Complete these 30 minutes before the call. Every item must be green before you start.

- **Terminal 1 (Server):** `cd ~/fintech-demo && git checkout main && git restore . && rm -f payments.db && python3 -m uvicorn app.main:app --reload --port 8000`
- **Terminal 2 (Demo):** `cd ~/fintech-demo && claude` → then type `/fast`
- **Browser:** Open `http://localhost:8000` — position beside terminal for screen share

Verify on Dashboard:

- 25 seeded transactions visible
- Amounts show **\$9,900** (bug is active — `formatAmount` missing `/100`)
- Success rate shows **99.2%**
- “Simulate P1 Incident” button is **red and enabled**
- 3 failed transactions visible (red badges)
- Claude Code open in Terminal 2, **fresh session**, `/fast` ON
- This guide open in a separate window (not a terminal tab)

TIP

If amounts show correctly at start: The `formatAmount` bug was fixed from a prior run. Run `git restore .` and restart the server. **If the table is empty:** Old `payments.db` wasn't deleted. `rm -f payments.db` and restart.

Demo Sequence

Setup → Prompt 1 → Prompt 2 → REFRESH → P1 BUTTON → Prompt 3 → Prompt 4
→ Dashboard resolves → Prompt 5 → PR banner → CLAUDE.md close

0

Setup Narration

SDLC: FRAMING

0:00 – 0:30 | 30 seconds | You talk, don't touch Claude

ACTION: Switch to browser. Point at dashboard.

SAY

"Let me set the scene. This is a real payments service — FastAPI backend, SQLite, external processor integration. Day one for a new engineer. They've just been given access to the codebase and the dashboard."

ACTION: Point at the \$9,900 amounts.

SAY

"First thing they notice: something's off."

1

Codebase Understanding

SDLC: UNDERSTAND

0:30 – 2:30 | 2 minutes | Claude runs ~35 sec

TYPE THIS PROMPT:

Walk me through this codebase — what does it do, how does money move through it, what should I know before I touch anything?

TIP

Watch for: 6–8 Read calls (main.py, payments.py, processor.py, database.py, config.py, models.py). No file paths in the prompt — Claude found them autonomously.

WHILE CLAUDE RUNS (~35 sec):

SAY

"Watch the tool calls. Read on main.py, payments.py, processor.py, database.py. It's reading the entire service — routes, business logic, the external processor integration, the database schema. No file paths in the prompt. It found them."

"This is how Claude Code works as an agent. It doesn't wait for you to open files. It uses Read, Grep, Glob tools to explore the codebase autonomously — the same way a senior engineer would cd around a new repo."

AFTER RESPONSE:

SAY

"Payment flow end to end. Processor isolation pattern. Security issues it flagged unprompted. What normally takes a new engineer a week of onboarding — meetings, Slack, wiki pages — took 30 seconds."

"And this works the same way whether Claude Code is running in the terminal, in VS Code, or in JetBrains. Same agent, same capabilities. Your team picks their environment."

2

Visual Bug Fix

SDLC: BUILD

2:30 – 5:00 | 2.5 minutes | Claude runs ~30 sec

ACTION: Point at \$9,900 amounts on dashboard.

SAY

"Every transaction is 100x the actual amount. In a payments dashboard, that's alarming."

TYPE THIS PROMPT:

Dashboard is showing amounts 100x too high – a \$99 charge shows as \$9,900. Find and fix it.

TIP

Watch for: Grep for formatAmount → Read index.html → Edit the exact line. No file name in the prompt — described it like a Slack message from product.

WHILE CLAUDE RUNS (~30 sec):

SAY

"No file name. No line number. Described it the way a Slack message from product would read. Watch — it's about to Grep for the formatting function, Read the file, and Edit the exact line."

"This is the difference between autocomplete and an agent. Copilot needs you to have the right file open. Claude Code searches the codebase, finds formatAmount() in index.html, identifies that it's passing raw cents without dividing by 100, and writes the one-line fix."

AFTER RESPONSE:

SAY

"Found it. static/index.html, the formatAmount function. Amounts stored in cents, displayed as dollars. One line — cents / 100."

ACTION: REFRESH THE BROWSER NOW. Amounts now correct.

SAY

"Fixed. The server picked up the change automatically. For 180 engineers, that's the difference between a 20-minute debugging session and 30 seconds."



Trigger P1 Incident

SDLC: TRANSITION

5:00 – 5:15 | 15 seconds

SAY

"Now let me show you what keeps your SREs up at night."

ACTION: CLICK "SIMULATE P1 INCIDENT" BUTTON NOW.

Red P1 banner appears. Success rate drops 99.2% → 94.1%. Duplicate charge rows appear.

SAY

"Payment success rate just cratered. Customers are being double-charged."



P1 Diagnosis

SDLC: TEST / DIAGNOSE

5:15 – 7:45 | 2.5 minutes | Claude runs ~45 sec

TYPE THIS PROMPT:

P1 – success rate dropped from 99.2% to 94.1%, duplicate charges hitting same customers. Diagnose root cause.

TIP

Watch for: Multiple Read calls — payments.py, processor.py, cross-referencing with CLAUDE.md. This is multi-file reasoning, not pattern matching.

WHILE CLAUDE RUNS (~45 sec) — your technical depth moment:

SAY

"This is what your on-call SRE does at 2:47am when this alert fires. Except instead of spending 20 minutes reading files they've never seen, they have an agent that reads the entire payment path in seconds."

"Watch the tool calls — it's reading payments.py, then processor.py, then cross-referencing with CLAUDE.md to understand what the architecture should look like versus what it actually does. That's multi-file reasoning."

"Think about the scale. 180 engineers, dozens of services. Claude Code understands the dependency chain across your whole system. That's what turns a junior engineer into someone who can triage a P1."

AFTER RESPONSE — walk through root cause chain:

SAY

"Two bugs, one incident. First: processor.py — every call to the external processor generates a fresh random UUID as the idempotency key. Retries send a different key. The processor treats it as a new charge. Second: payments.py — no error handling around the processor call. A timeout becomes an unhandled 500, the client retries, double charge."

"Claude didn't just find one bug. It traced the full failure chain across two files and explained how they interact. That's the difference between 'I found a bug' and 'I understand why your customers were double-charged.'"

4

P1 Fix + Tests

SDLC: BUILD + TEST

7:45 – 10:00 | 2.5 minutes | Claude runs ~45 sec

TYPE THIS PROMPT:

Fix it. Make sure all tests pass.

TIP

Watch for: Edit on processor.py (stable idempotency key) → Edit on payments.py (error handling) → Bash(pytest) — Claude decides to run tests itself.

WHILE CLAUDE RUNS (~45 sec):

SAY

"Five words. Watch what happens. It's about to edit processor.py — stable idempotency key, error classification. Then payments.py — reorder payment ID generation, wrap processor call in error handling. Then it runs pytest itself."

"That Bash tool call running pytest — Claude decided to do that. Not a hook. Not a script. The agent knows that in a regulated codebase, the fix isn't done until the tests pass. Same engineering rigor your senior engineers apply."

AFTER RESPONSE — 13 tests pass:

SAY

"13 tests. All green. The server reloaded. Watch the dashboard."

ACTION: PAUSE. Watch the dashboard. Stay silent. Within 3 seconds: P1 banner clears, success rate animates back to 99.2%.

SAY

"P1 resolved. Dashboard picked it up. No manual refresh, no deployment script."

Let it land.

5

Push PR for Review

SDLC: REVIEW + DEPLOY

10:15 – 12:15 | 2 minutes | Claude runs ~25 sec

SAY

"Fix works. Tests pass. In a regulated environment, you don't merge to main without review."

TYPE THIS PROMPT:

Push a PR for this fix so the team can review it.

TIP

Watch for: Bash(`git checkout -b`) → Bash(`git add`) → Bash(`git commit`) → Bash(`git push`) → Bash(`gh pr create`) → dashboard API call. Six shell commands from one prompt.

WHILE CLAUDE RUNS (~25 sec) — the agent power moment:

SAY

"This is where Claude Code is fundamentally different from any other AI coding tool. Watch the tool calls."

"Branch, stage, commit, push, open PR, notify. All from 'push a PR.' That's what agentic means — Claude Code doesn't just write code. It operates across your entire toolchain. Git, GitHub CLI, shell commands, API calls. It's an extension of your engineering team with unlimited capacity."

Green PR banner appears on dashboard within 3 seconds.

SAY

"The dashboard picked it up. Your team can see there's an open PR before anyone checks Slack. Click the link — that's a real PR on GitHub."

6

Governance Close

SDLC: COMPLIANCE

12:15 – 13:30 | 75 seconds | You talk, no Claude

SAY

"The last thing I want to show you isn't a prompt."

ACTION: Open CLAUDE.md in the terminal:

```
cat CLAUDE.md
```

FOR THE CTO:

SAY

"This file — CLAUDE.md — is committed to the repo. It encodes your PCI-DSS requirements, authentication patterns, the idempotency standard we just fixed, testing rules, SQL injection prevention. Plain English. Every engineer who opens Claude Code on this repo reads this file automatically. New hire, contractor, offshore team — same guardrails. Day one."

"This is the answer to your compliance question. Your engineering standards aren't in a Confluence page nobody reads. They travel with the codebase, and they're enforced automatically."

FOR THE HEAD OF DT:

SAY

"When you roll this out to 180 engineers, you don't train each of them on your security requirements. You write them once. The tool enforces them."

Timing Reference

Segment	Clock	Claude?	Tool Calls to Watch
Setup narration	0:00–0:30	No	—
Prompt 1: Understand	0:30–2:30	Yes (~35s)	Read × 6–8, Grep
Prompt 2: Build	2:30–5:00	Yes (~30s)	Grep → Read → Edit
Click P1	5:00–5:15	No	—
Prompt 3: Diagnose	5:15–7:45	Yes (~45s)	Read × 3, cross-file analysis
Prompt 4: Build + Test	7:45–10:00	Yes (~45s)	Edit × 2 → Bash(pytest)
Dashboard resolves	10:00–10:15	No	Stay silent
Prompt 5: Review	10:15–12:15	Yes (~25s)	Bash(git) × 4, Bash(gh pr), API call
Governance close	12:15–13:30	No	cat CLAUDE.md

While Claude Runs — Talking Points

These are your 30–45 second windows. Don't fill every second — silence while tool calls scroll is powerful.

For the CTO (security, control, compliance):

- “Every tool call is logged. Enterprise tier gives you full audit trails — what Claude read, edited, ran.”
- “Claude Code runs in terminal, VS Code, and JetBrains. Same agent. Governance is environment-agnostic.”
- “The Edit tool does targeted string replacement — cleaner diffs, easier code review, fewer merge conflicts.”
- “Notice Claude using parameterized SQL queries. It read CLAUDE.md and applied it. That's the governance layer.”

For the Head of DT (scale, velocity, team extension):

- “180 engineers, each saving 30 min/day on comprehension and debugging. That's 90 engineer-hours/day.”
- “The PR Claude created — a human reviews it. But they review a well-structured diff, not spend an hour writing it.”
- “Junior engineers operate at the level of seniors. Seniors operate at the level of a team.”
- “Every new hire inherits CLAUDE.md guardrails on day one. No onboarding lag for engineering standards.”

If Claude is taking longer than expected:

- “Complex reasoning takes a moment. Same way a senior engineer pauses to think through a multi-file change.”
- Point at tool calls: “You can see it working — this isn't a timeout. It's doing the work.”

If Things Go Wrong

Problem	Fix	What to Say
Claude response very long	Scroll to the summary	"It found the issue — here's the key part."
Dashboard doesn't update	Manual refresh (Cmd+R)	"Server reloaded — let me refresh."
P1 button greyed out	Ctrl+C → rm -f payments.db → restart	"Let me reset the environment."
Amounts correct at start	git restore . and restart server	—
Table is empty	rm -f payments.db → restart	—
PR banner doesn't appear	Wait 3 sec (poll cycle)	"Dashboard polls every 3 sec — there it is."
Claude gives wrong answer	Rephrase once	"Let me tighten that prompt."
Claude hangs	Press Escape, rephrase	"Let me be more specific."
Server died	Restart uvicorn in Terminal 1	"Let me restart the server."
Port 8000 in use	lsof -ti :8000 xargs kill	—

Quick Reset Between Runs

```
/exit
cd ~/fintech-demo
gh pr close $(gh pr list --json number -q '.[0].number') 2>/dev/null
git checkout main
git restore .
git push origin --delete fix/ftc-4421-duplicate-charges 2>/dev/null
git branch -D fix/ftc-4421-duplicate-charges 2>/dev/null
rm -f payments.db
lsof -ti :8000 | xargs kill 2>/dev/null
python3 -m uvicorn app.main:app --reload --port 8000 &
claude # then type /fast
```

All Five Prompts — Copy-Paste Ready

PROMPT 1 — UNDERSTAND

Walk me through this codebase – what does it do, how does money move through it, what should I know before I touch anything?

PROMPT 2 — BUILD

Dashboard is showing amounts 100x too high – a \$99 charge shows as \$9,900. Find and fix it.

PROMPT 3 — DIAGNOSE

P1 – success rate dropped from 99.2% to 94.1%, duplicate charges hitting same customers. Diagnose root cause.

PROMPT 4 — BUILD + TEST

Fix it. Make sure all tests pass.

PROMPT 5 — REVIEW + DEPLOY

Push a PR for this fix so the team can review it.